

Ideen zum Verwaltungswerkzeug

Gesamtzusammenstellung aller Funktionen — mit technischer Bewertung

Version: 1.0 · Datum: 2026-04-14

Grundlage: Brainstorming-Ergebnis (undatiert) + Bewertung gegen bestehende Architektur

Referenzdokumente: `Architektur_Interfaces_v0_3_build003.md`,
`Bauplan_Baustelle2_Webserver_v0_4.md`, `TP-A_Nutzerdaten_Tabellen_URLs_v0_34.md`

Klassifikation: VERTRAULICH — NUR FÜR DEN DIENSTGEBRAUCH

Vorbemerkung: Technische Ausgangslage

Baustelle 7 (Management-Interface) hat direkten `coordinator.db`-Zugriff und ist laut Architektur-Dokument **unabhängig von Baustelle 2** als Laufzeitsystem. Die offene Frage OP-H2 (CLI vs. Web) ist für alle nachfolgenden Ideen vorläufig als **Web** beantwortet: Die meisten Ideen erfordern interaktive Visualisierungen (Ampel, Matrix, Gantt, Zeitstrahl), die per CLI nicht sinnvoll darstellbar sind.

Datenbasis `coordinator.db` (aktueller Stand):

Tabelle	Relevante Spalten
<code>scrape_jobs</code>	<code>user_id</code> , <code>username</code> , <code>priority</code> , <code>status</code> , <code>assigned_to</code> , <code>created_at</code> , <code>started_at</code> , <code>finished_at</code> , <code>error_message</code>
<code>investigators</code>	<code>id</code> , <code>system_username</code> , <code>display_name</code> , <code>is_investigator</code> , <code>is_supervisor</code> , <code>is_support</code> , <code>created_at</code>

Implementierungsreihenfolge (Empfehlung): Baustelle 7 ist gemäß Gesamtplan nach Baustelle 5 vorgesehen. Die Bewertungen unten berücksichtigen, welche Daten zu diesem Zeitpunkt bereits existieren.

Bewertungsmatrix

#	Idee	Umsetzbarkeit	Aufwand	Datenbasis vorhanden?	Priorität
1	Echtzeit-Ampel-Dashboard	✓ Hoch	Mittel	✓ Ja	P1 — Kern
2	Statistik-Export für StA	✓ Hoch	Mittel	✓ Ja	P2

#	Idee	Umsetzbarkeit	Aufwand	Datenbasis vorhanden?	Priorität
3	Risiko-/Dringlichkeits-Matrix	⚠️ Bedingt	Hoch	⚠️ Teilweise	P3 — Datenlücke
4	Differenzierungsansicht Berichtsfreigabe	✅ Hoch	Hoch	✅ Ja (B6-abhängig)	P2 — nach B6
5	Gantt-Chart Ressourcenplanung	⚠️ Bedingt	Sehr hoch	❌ Nein	Zurückstellen
6	Lastverteilungs-Dashboard	✅ Hoch	Mittel	⚠️ Teilweise	P2
7	Prognose-Modul (3 Szenarien, PDF)	⚠️ Bedingt	Sehr hoch	❌ Nein	Zurückstellen
8	Ermittler-Metriken & Ausreißer	✅ Hoch	Mittel	✅ Ja	P2
9	Internes Nachrichtensystem	✅ Hoch	Gering	⚠️ Neue Tabelle	P2
10	Backup & Point-in-Time-Recovery	✅ Hoch	Mittel	✅ Ja (B1-koordiniert)	P1 — operativ kritisch
11	Fall-Ereigniszeitstrahl	✅ Hoch	Gering	✅ Ja (neue Tabelle)	P1
12	StA-Export-Modul	✅ Hoch	Mittel	✅ Ja (B6-abhängig)	P2 — nach B6
13	Audit-Log (kryptografische Verkettung)	✅ Hoch	Gering	⚠️ Neue Tabelle	P1 — forensisch essenziell
14	Volltextsuche über alle User-IDs	⚠️ Bedingt	Sehr hoch	✅ Ja (Skalierungsproblem)	v2.0
15	Modul-Steuerung / rollenbasierte Widgets	✅ Hoch	Mittel	✅ Ja	P3 — UX

Prioritätsstufen: P1 = in v1.0 zwingend · P2 = in v1.0 sinnvoll · P3 = optional · Zurückstellen = v1.1 oder v2.0

Im Folgenden alle besprochenen Punkte in Kurzform + Prosa + Bewertung:

1. Echtzeit-Ampel-Dashboard (pro User-ID)

Kurzform: Visueller Status (rot/gelb/grün) jeder User-ID mit Fortschrittsbalken und bearbeitendem Ermittler.

Prosa: Die Chefermittlerin benötigt auf einen Blick den Überblick über Dutzende parallel laufende Ermittlungen. Die Ampel zeigt: Rot (nicht begonnen), Gelb (in Arbeit mit Namen des Ermittlers und letzter Aktivität), Grün (abgeschlossen, Freigabe liegt vor). Der Fortschrittsbalken basiert auf dem Verhältnis ausgewerteter Posts/PMs zur Gesamtzahl – so ist sofort erkennbar, ob ein Fall stockt.

Bewertung: Vollständig umsetzbar mit bestehender Datenbasis. `scrape_jobs.status` liefert Rot/Gelb-Zustand direkt; Grün erfordert eine neue Spalte `approved_at` oder einen Status-Wert `approved` in `scrape_jobs.status` (CHECK-Constraint erweitern). Der Fortschrittsbalken ist die kritische Stelle: Das Verhältnis „ausgewerteter Posts/PMs zur Gesamtzahl“ ist nicht in `coordinator.db` verfügbar — dieser Wert müsste aus `evidence_*.db` pro User aggregiert werden (Anzahl `page_visits` oder `annotations` gegen eine Gesamtzahl aus `scrape_jobs`). Empfehlung: Eine dedizierte Spalte `total_pages_scraped` in `scrape_jobs` beim Abschluss von Stage 2 befüllen; `evidence_db` liefert dann den Zähler für bereits besuchte Seiten. Ohne diesen Mechanismus kann kein echter Fortschrittsbalken berechnet werden. Der SSE-Mechanismus aus Baustelle 2 (Support-Indikator in Sektion 5 der Toolbar) kann als Vorbild für die Echtzeit-Aktualisierung des Dashboards dienen — die Infrastruktur ist bereits skizziert.

2. Automatische Statistik für die Staatsanwaltschaft (Export)

Kurzform: Anzahl laufender/abgeschlossener Fälle, durchschnittliche Bearbeitungsdauer, Heatmap der Aktivitätszeiten (pro Fall im Detail).

Prosa: Die Staatsanwaltschaft muss Verjährungsfristen im Blick behalten und Anklagezeiträume planen. Das Modul liefert aggregierte Kennzahlen: Wie viele Fälle sind diese Woche abgeschlossen worden? Wie lange dauert ein Fall durchschnittlich von Zuweisung bis Freigabe? Zudem kann sie für jeden abgeschlossenen Fall eine Aktivitätsheatmap anfordern, die Lebensrhythmen des Beschuldigten sichtbar macht.

Bewertung: Die Kennzahlen (Fallzahlen, Bearbeitungsdauer) sind direkt aus `scrape_jobs` (Timestamps `created_at`, `started_at`, `finished_at`) ableitbar. Die Aktivitätsheatmap erfordert jedoch Klarheit über die Datenquelle: Sind die Aktivitätszeiten des *Beschuldigten* (aus den Forum-Post-Timestamps in MariaDB) oder die Aktivitätszeiten der *Ermittler* (aus `page_visits` in `evidence_*.db`) gemeint? Beides ist sinnvoll, dient aber grundlegend unterschiedlichen Zwecken. Die Beschuldigten-Heatmap gehört forensisch in den Nutzerinfo-Tab (Baustelle 4), nicht in den StA-Export — dort sollte sie als Extrakt erscheinen. Für den Export-Mechanismus gilt: Das Format des Exports (PDF, CSV, beides?) muss mit der Staatsanwaltschaft abgestimmt sein. Eine Abhängigkeit von Baustelle 6 (Bericht) besteht für den strukturierten Teil; reine Statistiken können jedoch früher exportiert werden. Achtung: Da das Netzwerk offline ist, muss das Exportverzeichnis physisch erreichbar sein (USB,

Netzlaufwerk) — das ist kein Python-Problem, aber ein organisatorischer Ablauf, der im Werkzeug dokumentiert werden muss.

3. Risiko- & Dringlichkeits-Matrix

Kurzform: 3x3-Raster mit verbleibenden Tagen bis Verjährung (X) vs. Relevanz-Score (Y) – jeder User ist ein klickbarer Punkt.

Prosa: Nicht alle Fälle sind gleich eilig. Die Matrix kombiniert zwei kritische Dimensionen: Wie knapp ist das Zeitfenster vor Verjährung? Wie hoch ist die inhaltliche Relevanz des Users (basierend auf Sprache, Aktivitätszeiten, Passwortauffälligkeiten)? Daraus ergeben sich klare Handlungsempfehlungen wie „Sofort priorisieren“ oder „Dokumentieren und zurückstellen“. Ein Klick auf einen Punkt führt direkt zum Fall.

Bewertung: Die Idee ist konzeptionell wertvoll, aber die Datenbasis ist in zwei kritischen Punkten nicht vorhanden. Erstens: Verjährungsfristen sind nirgendwo im System erfasst — weder in `coordinator.db` noch in `scrape_jobs`. Diese müssen pro User-ID manuell gepflegt werden; dafür ist eine neue Spalte `limitation_deadline DATE` in `scrape_jobs` (oder einer neuen `cases`-Tabelle) erforderlich. Die Verjährungsfrist für §§ 176a, 184b StGB beträgt gemäß § 78 StGB 20 Jahre, kann aber durch Verfolgungsverjährung, Ruhenszeiten (§ 78b StGB) oder individuelle Tatzeiten variieren — eine automatische Berechnung ist rechtlich nicht zulässig, nur manuelle Eingabe durch die Ermittlungsleitung. Zweitens: Der „Relevanz-Score“ auf der Y-Achse ist das konzeptionell schwächste Element. Welche Faktoren fließen ein, mit welcher Gewichtung, und wer validiert den Score? Ein algorithmisch generierter Relevanz-Score, der zur Priorisierung oder Zurückstellung von Verdächtigen führt, hat unmittelbare rechtliche Implikationen (Gleichbehandlungsgrundsatz, § 261 StPO zur freien Beweiswürdigung). Empfehlung: Die Y-Achse als manuell einzuschätzende Priorität (1-5, bereits in `scrape_jobs.priority` vorhanden) belassen, keine automatische Score-Berechnung vornehmen. Die Matrix bleibt so nützlich, ohne rechtlich angreifbar zu sein.

4. Differenzierungsansicht für Berichtsfreigabe

Kurzform: Side-by-Side-Ansicht (Originalbericht vs. Überarbeitungsvorschlag) mit Prinzip der unveräußerbaren Autorschaft.

Prosa: Mehrere Ermittler arbeiten kollaborativ an einem Fallbericht – jeder Absatz bleibt seinem Autor zugeordnet und kann von anderen nur kommentiert, nicht überschrieben werden. Die Chefermittlerin sieht links den kollaborativen Bericht (farbig markiert nach Autoren), rechts ihren Änderungsvorschlag. Jede ihrer Formulierungen wird als *Suggestion* gespeichert. Die finale Freigabe erzeugt einen kryptografisch signierten Vermerk in der `coordinator.db`.

Bewertung: Konzeptionell solide und für die Gerichtsverwertbarkeit von hoher Bedeutung — die unveräußerliche Autorschaft ist kein UX-Feature, sondern ein Beweissicherungsmerkmal. Die Datenbasis ist in Baustelle 4 bereits vorgesehen: `report_paragraphs`, `report_anchors`, `report_suggestions`, `report_approvals` und `editor_locks` sind im Schema von

`evidence_<uid>.db` definiert. Die Implementierung dieser Ansicht gehört jedoch primär in Baustelle 6 (Bericht), nicht in Baustelle 7; Baustelle 7 übernimmt nur den Freigabe-Workflow (Signierschritt in `coordinator.db`). Kritisch zu beachten: Der `editor_locks`-Mechanismus muss Deadlocks verhindern — wenn ein Ermittler einen Absatz sperrt und die Verbindung verliert, muss ein Timeout-Mechanismus die Sperre automatisch freigeben. Das ist technisch nicht trivial auf einem SMB-Netzlaufwerk. Ferner: Das kollaborative Arbeiten an einem Bericht mit mehreren gleichzeitigen Autoren auf einem SQLite-File auf einem Netzlaufwerk (WAL-Mode + Retry) ist unter hoher Last fragil — hier ist die Anzahl gleichzeitiger Schreibzugriffe zu begrenzen (maximal ein aktiver Autor pro `report_paragraph` erzwingen).

5. Gantt-Chart mit Ressourcenplanung

Kurzform: Zeitliche Darstellung aller Ermittler, ihrer Verfügbarkeiten und zugewiesenen User-IDs mit Meilensteinen.

Prosa: Die Chefermittlerin muss gegenüber der Staatsanwaltschaft belastbare Prognosen liefern können. Das Gantt-Chart zeigt auf einer Zeitachse, welcher Ermittler wann an welchem Fall arbeitet, basierend auf hinterlegten Verfügbarkeitskalendern. Meilensteine wie „Zwischenbericht an StA“ oder „Freigabe durch Chefermittlerin“ strukturieren den Prozess zusätzlich.

Bewertung: Die Idee ist operativ sinnvoll, aber die notwendige Datenbasis fehlt vollständig. Ein Gantt-Chart setzt voraus: hinterlegte Verfügbarkeitskalender pro Ermittler (Urlaub, Teilzeit, andere Projekte), Schätzwerte für den Aufwand pro User-ID und historische Velocity-Daten (wie lange hat Ermittler X bisher für vergleichbare Fälle benötigt). Keine dieser Daten ist im System vorhanden oder kann automatisch ermittelt werden. Eine Verfügbarkeits-Tabelle (`investigator_availability`) müsste manuell gepflegt werden, was einen erheblichen organisatorischen Aufwand bedeutet. Empfehlung: Zurückstellen auf v1.1. In v1.0 kann eine einfachere Variante angeboten werden: ein tabellarischer Überblick mit `assigned_to`, `started_at` und der manuell gesetzten Priorität, ohne Zeitachsenprojektion. Das liefert 80 % des Nutzens bei 20 % des Aufwands.

6. Lastverteilungs-Dashboard mit Überlastwarnung

Kurzform: Berechnung offener Stunden vs. verfügbarer Ermittlerstunden, Warnung bei Überlast, Vorschlag zur Umverteilung.

Prosa: Wenn die Summe der geschätzten Restaufwände die verfügbaren Kapazitäten übersteigt, erscheint eine Warnung mit der genauen Differenz. Das Dashboard schlägt konkrete Umverteilungen vor: „Ermittlerin A hat eine Lücke in ihrem Kalender – ihr könnte User-ID 42 übernehmen.“ So wird die Chefermittlerin proaktiv unterstützt, bevor es zu Verzögerungen kommt.

Bewertung: Der qualitative Teil (wer hat wie viele Fälle zugewiesen, wer hat keine) ist sofort aus `scrape_jobs.assigned_to` ableitbar und nützlich. Der quantitative Teil (Stunden vs.

Kapazitäten) hat dasselbe Problem wie Idee 5: Stundenaufwände sind nicht im System erfasst. Pragmatische Lösung: Die Anzahl zugewiesener, noch nicht abgeschlossener Fälle pro Ermittler als einfachen Lastindikator verwenden — kein Stunden-Model, aber aussagekräftig. Die „Vorschläge zur Umverteilung" sollten als Hinweis, nicht als automatische Aktion implementiert werden: „Ermittlerin A hat 0 laufende Fälle" ist eine Tatsache, die Entscheidung über Zuweisung trifft ausschließlich die Chefermittlerin. Keine automatische Zuweisung.

7. Prognose-Modul für die Staatsanwaltschaft (drei Szenarien)

Kurzform: Optimistische, realistische und pessimistische Fertigstellungsprognose exportierbar als PDF mit Wasserzeichen.

Prosa: Die Staatsanwaltschaft benötigt Planungssicherheit für Anklageerhebungen. Das Modul berechnet auf Basis historischer Bearbeitungszeiten und aktueller Verfügbarkeiten drei Szenarien: optimistisch (keine Störungen), realistisch (80 % Verfügbarkeit, 15 % Puffer) und pessimistisch (Ausfälle, Rückfragen). Das PDF erhält ein vorläufiges Wasserzeichen und wird regelmäßig aktualisiert.

Bewertung: Das Modul ist in v1.0 nicht umsetzbar — nicht wegen technischer Komplexität, sondern wegen fehlender Kalibrierungsdaten. Statistische Prognosemodelle (auch einfache lineare) benötigen historische Basisdaten zur Kalibrierung. Zu Projektbeginn existieren keine. Ein Modell, das ohne Kalibrierung eine Prognose an die Staatsanwaltschaft ausgibt, ist schlimmer als keine Prognose — es erzeugt falsches Vertrauen. Empfehlung: Das Modul ab ca. 30-50 abgeschlossenen Fällen aktivieren, wenn ausreichend Velocity-Daten vorliegen. Der PDF-Export mit Wasserzeichen ist technisch einfach (ReportLab oder WeasyPrint); das Wasserzeichen „VORLÄUFIG" ist zwingend, solange die Kalibrierungsbasis klein ist. Das Modul sollte bei jedem Export die Anzahl der Kalibrierungsfälle im Footer ausweisen — das ist Dokumentationspflicht.

8. Ermittler-Metriken & Ausreißer-Detektion (Qualitätssicherung)

Kurzform: Automatische Erkennung von Ermittlern mit Abweichungen vom Median bei Aktivität, Annotationstiefe oder Rückweisungsquote.

Prosa: Um die Beweisqualität über das gesamte Projekt zu gewährleisten, müssen Auffälligkeiten früh erkannt werden. Das Dashboard zeigt pro Ermittler Kennzahlen wie bearbeitete Einheiten pro Stunde, durchschnittliche Länge von Anmerkungen und das Verhältnis freigegebener zu zurückgewiesener Berichte. Wer signifikant unter oder über dem Median liegt, wird automatisch markiert.

Bewertung: Technisch gut umsetzbar; die Datenbasis (`page_visits`), (`annotations`), (`report_approvals`) in (`evidence_*.db`) existiert nach Abschluss von Baustellen 3-5. Ein wesentlicher Hinweis zur forensischen Integrität: Die Metriken dürfen ausschließlich der Chefermittlerin sichtbar sein, niemals den betroffenen Ermittlern selbst. Wenn ein Ermittler weiß, dass seine Annotationslänge gemessen wird, entsteht ein Anreiz zum „Padding"

(aufgeblähte, inhaltlich dünne Anmerkungen), was die Beweisqualität aktiv schädigt. Die Ausreißer-Erkennung sollte keine Automatik auslösen (kein automatisches Flag in der Beweisdatei), sondern nur der Chefermittlerin als Entscheidungsgrundlage für ein Gespräch dienen. Datenschutzrechtlich ist zu prüfen, ob die Erhebung von Leistungsmetriken über beamtete Ermittler im NRW-Dienstrecht zulässig ist — das ist keine Python-Frage, aber eine Frage, die vor Implementierung mit der Ermittlungsleitung geklärt werden sollte.

9. Internes Nachrichtensystem (nur standardisierte Templates)

Kurzform: Einbahnstraße von der Chefermittlerin zu Ermittlern mit vordefinierten Textbausteinen, keine personenbezogenen Daten.

Prosa: Im abgeschotteten Netzwerk stehen keine E-Mail- oder Chat-Systeme zur Verfügung. Das interne Nachrichtensystem erlaubt der Chefermittlerin dennoch, standardisierte Hinweise zu versenden – z. B. „Bitte erhöhen Sie die Annotationstiefe“ oder „Ihre Geschwindigkeit liegt unter dem Median“. Die Nachricht erscheint als Banner im Arbeitsplatz des Ermittlers. Freitext ist nicht erlaubt, um Datenschutzprobleme zu vermeiden.

Bewertung: Einfachste Implementierung des gesamten Katalogs. Eine neue Tabelle `notifications` in `coordinator.db` (Spalten: `id`, `recipient_id`, `template_id`, `sent_at`, `read_at`) reicht aus. Der Banner im Arbeitsplatz des Ermittlers kann über den SSE-Stream aus Baustelle 3 (Support-Indikator) mit sehr geringem Aufwand ergänzt werden — die Infrastruktur ist bereits vorhanden. Die Einschränkung auf Templates ist forensisch klug: Kein Freitext bedeutet kein Risiko unbeabsichtigter Verfahrensabsprachen im System-Log. Die Templates sollten versioniert und unveränderlich sein (Enum im Code, nicht in der DB) — damit ist sichergestellt, dass ein Template-Eintrag im Audit-Log eindeutig einer Formulierung entspricht, auch Jahre später.

10. Backup & Point-in-Time-Recovery-Maske

Kurzform: Nächtliche automatische Snapshots aller Datenbanken, Wiederherstellung mit Dry-Run und Bestätigung.

Prosa: Backups sind automatisiert, aber die Wiederherstellung muss auch von der Chefermittlerin selbst angestoßen werden können. Die Maske listet alle verfügbaren Backups mit Datum und Größe. Ein Dry-Run zeigt vor der eigentlichen Wiederherstellung, welche Änderungen rückgängig gemacht würden. Erst nach einer zweiten Bestätigung („Ich bin sicher“) werden die aktiven Datenbanken überschrieben.

Bewertung: Operativ kritisch und gut durchdacht. Wichtiger Hinweis zur Datenbankarchitektur: `forensic_<uid>.db` ist READ-ONLY und nach Baustelle-0-Definition aus den Originaldaten reproduzierbar — ein Backup davon ist nicht notwendig und sollte in der UI explizit als „nicht sicherungspflichtig“ markiert werden, damit kein falsches Sicherheitsgefühl entsteht. `evidence_<uid>.db` enthält alle Ermittlungsanmerkungen und ist das eigentliche Sicherungsgut. Der „Dry-Run“ ist für SQLite konzeptionell schwierig: SQLite

kennt keine echte Vorschau-Transaktion über Datenbankgrenzen hinweg. Der Dry-Run kann sinnvollerweise nur eine Auflistung der Datei-Zeitstempel und -Größen des Backups vs. der aktuellen Datei liefern — keine zeilenweise Diff-Anzeige. Das ist ehrlicher kommuniziert als ein vollständiger „was würde sich ändern“-Report. Koordination mit Baustelle 1 (Deployment-Paket) ist zwingend: Die Backup-Logik gehört in Baustelle 1, Baustelle 7 liefert nur die UI-Oberfläche dafür.

11. Fall-bezogener Ereigniszeitstrahl

Kurzform: Vertikale Timeline pro User-ID mit automatischen und manuellen Ereignissen (Zuweisungen, Statuswechsel, Rückstellungen).

Prosa: Die Kommunikation mit der Staatsanwaltschaft erfordert nachvollziehbare Entscheidungen. Der Zeitstrahl protokolliert automatisch jede Zuweisung und Statusänderung. Die Chefermittlerin kann manuell Einträge hinzufügen wie „Rückstellung bis 15.07.2026 – Grund: weitere Beschlüsse erforderlich“ oder „Priorität erhöht auf Anordnung der StA“. Das schafft Transparenz und dokumentiert die Ermittlungsführung.

Bewertung: Eine der wertvollsten Ideen im gesamten Katalog, mit dem geringsten Implementierungsaufwand. Eine neue Tabelle `case_events` in `coordinator.db` (Spalten: `id`, `user_id`, `event_type`, `event_text`, `author_id`, `ts`, `is_manual`) erfasst alle Ereignisse. Automatische Ereignisse (Statuswechsel in `scrape_jobs`) werden per Trigger oder beim Schreiben in `scrape_jobs` eingetragen; manuelle Einträge kommen aus dem Verwaltungs-Interface. Forensisch entscheidend: Manuelle Einträge dürfen nicht nachträglich editiert oder gelöscht werden. Es darf nur ein neuer Eintrag hinzugefügt werden, der einen früheren korrigiert — das Audit-Log (Idee 13) muss auch `case_events`-Schreibvorgänge erfassen. Der Zeitstrahl ist gleichzeitig die Grundlage für den StA-Export (Idee 12) und für die Verfahrensdokumentation — er sollte daher von Anfang an mitlaufen, auch wenn Baustelle 7 noch nicht fertig ist. Die Zeitstrahl-Befüllung gehört teilweise bereits in Baustelle 5.

12. StA-Export-Modul mit Ausschleus-Verzeichnis

Kurzform: Export von Berichten, Zeitstrahl und Beweismittelverzeichnissen in ein verschlüsseltes Transferverzeichnis.

Prosa: Das ermittlungsinterne Netzwerk ist vom Internet getrennt. Für die Übergabe an die Staatsanwaltschaft werden Exporte in ein spezielles Ausschleus-Verzeichnis geschrieben, verschlüsselt mit einem einmaligen Schlüssel. Ein sicherer Transferagent holt die Dateien ab. Nach erfolgreichem Abholen können die Quelldateien aus dem Arbeitsverzeichnis gelöscht werden – so bleibt das Arbeitsnetzwerk sauber.

Bewertung: Technisch solide konzipiert. Drei kritische Präzisierungen: Erstens muss der „einmalige Schlüssel“ genauer spezifiziert werden — gemeint ist vermutlich asymmetrische Verschlüsselung (Public Key der Staatsanwaltschaft), nicht ein einmaliger symmetrischer Schlüssel. Ein symmetrischer Einmalschlüssel erfordert einen sicheren Out-of-Band-Kanal zur

Schlüsselübergabe, der im abgeschotteten Netzwerk organisatorisch aufwändig ist. Zweitens: Das „Löschen nach erfolgreichem Abholen“ ist aus forensischer Sicht heikel. Wenn das Abholen durch den Transferagenten der einzige Nachweis der Übergabe ist, fehlt eine quitierte Übergabe. Empfehlung: Die exportierten Dateien werden nicht gelöscht, sondern in ein Archiv-Verzeichnis verschoben, mit Zeitstempel der Übergabe und einer Quittungs-ID im Audit-Log. Drittens: Der Export muss zwingend einen SHA-256-Hash der exportierten Dateien im Audit-Log hinterlassen — damit ist bei einer späteren Gerichtsverhandlung nachweisbar, was exakt zum Zeitpunkt X an die Staatsanwaltschaft übergeben wurde.

13. Audit-Log mit Nachweis der Nichtmanipulierbarkeit

Kurzform: Kryptographisch verkettete Log-Tabelle (Hash der vorherigen Zeile) für alle kritischen Aktionen.

Prosa: Für die spätere Gerichtsverwertbarkeit reicht ein einfacher SQLite-Timestamp nicht aus – ein Administrator könnte theoretisch Einträge nachträglich ändern. Das Audit-Log arbeitet wie eine Mini-Blockchain: Jeder neue Eintrag enthält den Hash des vorherigen Eintrags plus einen geheimen Salt. Die Chefermittlerin kann jederzeit die Integrität prüfen lassen. Weicht ein Hash ab, ist eine Manipulation nachweisbar.

Bewertung: Forensisch essenziell — dies ist das einzige Element des Katalogs, das keine Abwägung erfordert: Es muss implementiert werden, und zwar **vor** allen anderen Ideen. Technische Präzisierung: Der „geheime Salt“ in der Prosa ist missverständlich. Ein salt pro Eintrag ist sinnvoll, um Rainbow-Table-Angriffe auf die Hash-Chain zu erschweren. Der Salt sollte jedoch nicht geheim im Sinne von „irgendwo separat gespeichert“ sein — dann wäre er ein zweiter Angriffspunkt. Empfehlung: SHA-256(prev_hash || event_type || actor_id || ts || content) als deterministische Berechnung ohne separaten Salt; die Chain-Integrität ergibt sich aus der Verkettung selbst. Das ist etablierte Praxis und gerichtlich anerkannt. Wichtig: Das Audit-Log muss **alle** schreibenden Operationen auf `(coordinator.db)` erfassen, nicht nur „kritische Aktionen“ — die Grenze zwischen kritisch und unkritisch ist später nicht mehr rekonstruierbar. Ferner: Die `(forensic_<uid>.db)`-Signaturen (SHA-256, bereits im System vorhanden) und das `(coordinator.db)`-Audit-Log sind zwei verschiedene Sicherungsebenen und ergänzen sich — das sollte in der Dokumentation klar unterschieden werden.

14. Volltextsuche über alle User-IDs hinweg

Kurzform: Zentrale Suchmaschine über alle entschlüsselten Nachrichten und Posts, Treffer werden pro Fall gruppiert.

Prosa: Zusammenhänge zwischen verschiedenen Usern (gleiche Redewendungen, gleiche Orte, gleiche Fehler in der Verschlüsselung) entgehen sonst leicht. Die Volltextsuche erlaubt Queries wie „Nordbahnhof“ oder „treffen um 15 Uhr“ über alle `(evidence_*.db)` hinweg. Das Ergebnis zeigt sofort, welche User-IDs die Begriffe enthalten – und kann Prioritäten verschieben oder neue Ermittlungsansätze liefern.

Bewertung: Der investigative Mehrwert ist erheblich — Cross-User-Suche kann Verbindungen aufdecken, die in der Einzelfallbearbeitung unsichtbar bleiben. Das ist der stärkste Grund, diese Idee nicht aufzugeben, sondern sorgfältig zu planen. Die technische Herausforderung ist jedoch die bedeutendste des gesamten Katalogs: Bei potenziell Tausenden von `evidence_*.db`-Dateien auf einem Netzlaufwerk ist eine naive Volltextsuche (je Datei ATTACH + FTS5-Query) inakzeptabel langsam. Die einzig praktikable Architektur ist ein zentraler Suchindex: Ein Hintergrundprozess (Indexer-Daemon) liest neue Annotations- und PM-Inhalte aus fertiggestellten `evidence_*.db`-Dateien und schreibt sie in eine zentrale `search_index.db` mit SQLite FTS5. Suchanfragen gehen nur noch gegen `search_index.db`. Das ist umsetzbar, erfordert aber einen eigenen Daemon, eigenes Schema und eine Strategie für inkrementelle Index-Updates. Empfehlung: Architektur in v1.0 definieren und vorbereiten (Daemon-Schnittstelle, Schema), aber Implementierung auf v1.1 verschieben. Damit bleibt die Option offen, ohne v1.0 zu verzögern.

15. Modul-Steuerung mit Rollen-basierten Widgets (Chefermittlerin & Admin)

Kurzform: Konfigurierbares Dashboard mit Drag & Drop – Chefermittlerin wählt sichtbare Module, Admin sieht zusätzlich technische Metriken.

Prosa: Nicht jeder braucht alles. Die Chefermittlerin kann aus einer Modulbibliothek (Ampel, Gantt, Risikomatrix, Ausreißer-Monitor, Prognose-Export) die für ihre aktuelle Sitzung relevanten Widgets auswählen und per Drag & Drop anordnen. Der Administrator hat einen eigenen, nur für ihn sichtbaren Bereich mit SQLite-Connection-Status, ATTACH-Fehlern und aktiven Sessions.

Bewertung: Das rollenbasierte Sichtbarkeits-Modell ist sinnvoll und bereits in `investigators` (`is_supervisor`, `is_support`) verankert. Der Admin-Bereich (Connection-Status, ATTACH-Fehler, aktive Sessions) ist dabei das technisch wertvollere Element — ohne ihn hat niemand im laufenden Betrieb einen Überblick über den Systemzustand. Die Drag-and-Drop-Personalisierung ist dagegen ein Nice-to-Have: Sie bindet Implementierungsaufwand, der in v1.0 besser in die Kernmodule fließt. Empfehlung: In v1.0 feste, rollenbasierte Layouts implementieren (Chefermittlerin-View, Admin-View) ohne Drag-and-Drop; die Persistierung von Benutzereinstellungen (welches Widget ist sichtbar) kann über eine einfache `ui_preferences`-Tabelle in `coordinator.db` mit minimalem Aufwand ergänzt werden. Drag-and-Drop folgt in v1.1.

Zusammenfassung: Empfohlene Umsetzungsreihenfolge für v1.0

Sofort (P1 — forensisch oder operativ kritisch): 13 (Audit-Log) → 11 (Ereigniszeitstrahl) → 1 (Ampel-Dashboard) → 10 (Backup-Maske)

In v1.0 sinnvoll (P2): 9 (Nachrichtensystem) → 8 (Ermittler-Metriken) → 6 (Lastverteilung) → 2 (StA-Statistik) → 4 (Berichtsfreigabe, nach B6) → 12 (StA-Export, nach B6)

Optional / v1.1 (P3): 3 (Risikomatrix, nach Datenlücken-Lösung) → 15 (Modul-Steuerung) → 14 (Volltextsuche, Architektur vorbereiten)

Zurückstellen auf v1.1/v2.0: 5 (Gantt), 7 (Prognose-Modul)

Dokument-Ende · Bewertung Ideen Verwaltungswerkzeug · Version 1.0 · 2026-04-14 Internes Arbeitsdokument — IT-forensisches Ermittlungsprojekt NRW. Unterliegt der Vertraulichkeit des Ermittlungsverfahrens.